实验报告

胡皓然 计41 2024010671

1. 在模拟器中运行 RISC-V 监控程序，尝试term中命令的用法

在终端中启动 kernel:

```
❯ cd supervisor-rv
❯ cd kernel
❯ make sim
riscv64-unknown-elf-ld  obj/evec.o  obj/init.o  obj/shell.o  obj/test.o  obj/trap.o  obj/utils.o -Tkern/kernel32.ld
qemu-system-riscv32 -M virt -m 32M -kernel kernel.elf -nographic -monitor none -serial tcp::6666,server -s -bios none
qemu-system-riscv32: -serial tcp::6666,server: info: QEMU waiting for connection on: disconnected:tcp:0.0.0.0:6666,serve
r=on
```

开启另一个终端，启动 term:

```
❯ python3 term.py -t 127.0.0.1:6666
connecting to 127.0.0.1:6666...connected
MONITOR for RISC-V - initialized.
running in 32bit, xlen = 4
```

监控程序功能总结：

命令	功能	使用顺序示例
A	逐行输入汇编，写入指定内存地址	`A` → 地址 `80100000` → 逐行输入汇编 → **空行结束**
U	读取内存并反汇编，检查装入的指令	`U` → 地址 `80100000` → 字节数 `24`
G	从指定地址开始执行程序	`G` → 入口地址 `80100000`
R	查看用户程序结束时保存的寄存器值	输入 `R`，不需要参数
D	查看指定内存区域的数据	`D` → 地址 `80400000` → 字节数 `4`
F	从文件读取汇编并装入内存	`F` → 文件路径，如 `program.s` → 地址 `80100000`
Q	退出 Term	输入 `Q`；不会同时关闭 QEMU
T	查看页表	输入 `T`；

尝试常用功能：

1. 使用A写入指令：

```
>> A
addr: 0x80100000
one instruction per line, empty line to end.
[0x80100000] li t0, 1
[0x80100004] li t1, 2
[0x80100008] add t2, t0, t1
[0x8010000c] li t3, 0x80400000
[0x80100010] sw t2, 0(t3)
[0x80100014] jr ra
[0x80100018]
```

2. 使用U查看装入的指令：

```
>> U
addr: 0x80100000
num: 24
0x80100000:     00100293        li      t0,1
0x80100004:     00200313        li      t1,2
0x80100008:     006283b3        add     t2,t0,t1
0x8010000c:     80400e37        lui     t3,0x80400
0x80100010:     007e2023        sw      t2,0(t3)
0x80100014:     00008067        ret
```

3. 使用G运行指令：

```
>> G
addr: 0x80100000

elapsed time: 0.001s
```

4. 使用R查看寄存器：

```
>> R
R1 (ra)     = 0x800002cc
R2 (sp)     = 0x807f0000
R3 (gp)     = 0x00000000
R4 (tp)     = 0x00000000
R5 (t0)     = 0x00000001
R6 (t1)     = 0x00000002
R7 (t2)     = 0x00000003
```

5. 使用D查看内存：

```
>> D
addr: 0x80400000
num: 4
0x80400000: 0x00000003
```

2. 求前10个斐波那契数 并且保存到起始地址为 0x80400000 的 10 个字中

汇编代码：

```
    # t0 中保存当前结果存储地址
    li t0, 0x80400000
    # t1 中保存较旧的结果，t2 保存较新的结果，t3 保存刚刚算出来的结果，t4保存剩余循环次数
    li t1, 1
    li t2, 1
    li t4, 8
    sw t1, 0(t0)
    sw t2, 4(t0)
    addi t0, t0, 8
.L1:
    # 计算结果
    add t3, t1, t2
    # 存储结果
    sw t3, 0(t0)
    addi t0, t0, 4
    # 更新寄存器
```

```
    addi t1, t2, 0
    addi t2, t3, 0
    # 循环次数减一
    addi t4, t4, -1
    # 判断是否跳转
    bne zero, t4, .L1
    jr ra

```

过程截图：

1. 从文件加载程序：

```
>> F
>>file name: /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/fib10.s
>>addr: 0x80100000
reading from file /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/fib10.s
[0x80100000] # t0 中保存当前结果存储地址
[0x80100000] li t0, 0x80400000
[0x80100004] # t1 中保存较旧的结果，t2 保存较新的结果，t3 保存刚刚算出来的结果，t4保存剩余循环次数
[0x80100004] li t1, 1
[0x80100008] li t2, 1
[0x8010000c] li t4, 8
[0x80100010] sw t1, 0(t0)
[0x80100014] sw t2, 4(t0)
[0x80100018] addi t0, t0, 8
[0x8010001c] .L1:
[0x8010001c] # 计算结果
[0x8010001c] add t3, t1, t2
[0x80100020] # 存储结果
[0x80100020] sw t3, 0(t0)
[0x80100024] addi t0, t0, 4
[0x80100028] # 更新寄存器
[0x80100028] addi t1, t2, 0
[0x8010002c] addi t2, t3, 0
[0x80100030] # 循环次数减一
[0x80100030] addi t4, t4, -1
[0x80100034] # 判断是否跳转
[0x80100034] bne zero, t4, .L1
[0x80100038] jr ra
```

2. 查看写入的程序：

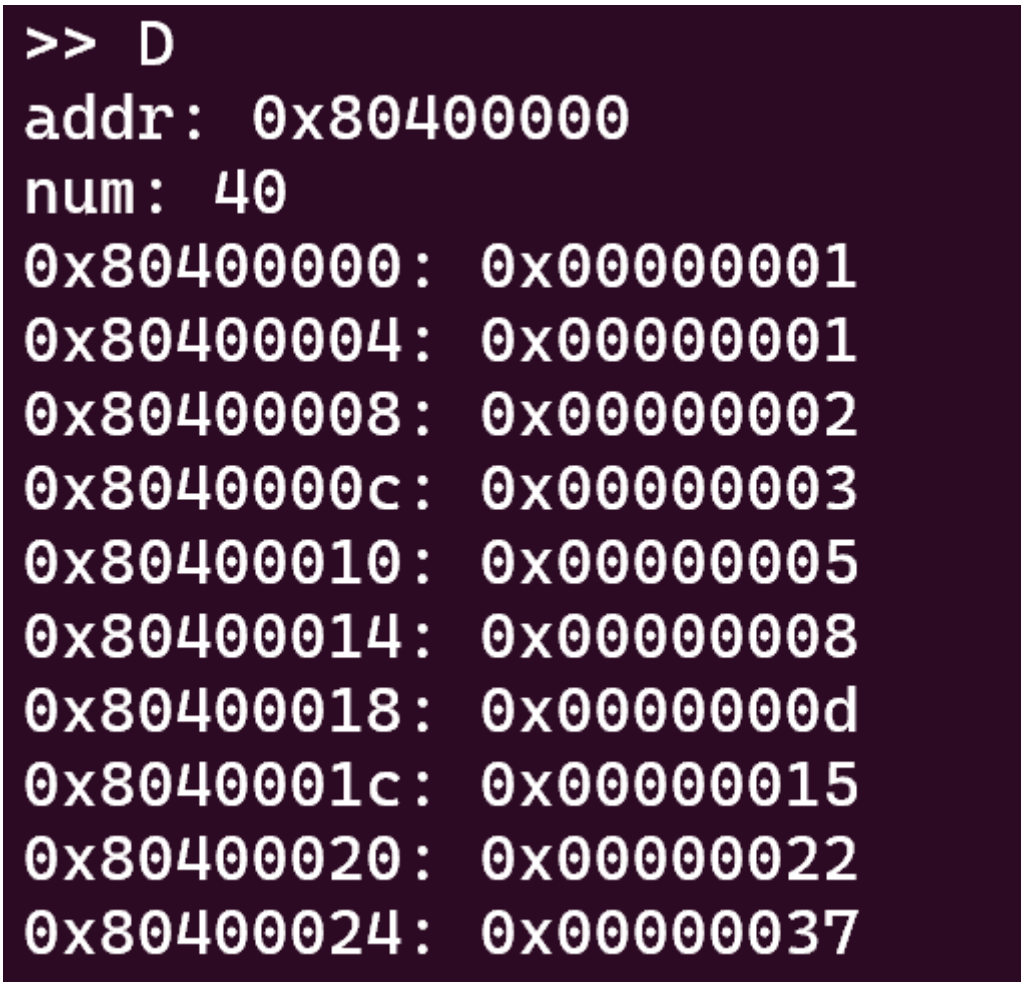
```
>> U
addr: 0x80100000
num: 60
0x80100000:     804002b7        lui     t0,0x80400
0x80100004:     00100313        li      t1,1
0x80100008:     00100393        li      t2,1
0x8010000c:     00800e93        li      t4,8
0x80100010:     0062a023        sw      t1,0(t0)
0x80100014:     0072a223        sw      t2,4(t0)
0x80100018:     00828293        addi    t0,t0,8
0x8010001c:     00730e33        add     t3,t1,t2
0x80100020:     01c2a023        sw      t3,0(t0)
0x80100024:     00428293        addi    t0,t0,4
0x80100028:     00038313        mv      t1,t2
0x8010002c:     000e0393        mv      t2,t3
0x80100030:     fffe8e93        addi    t4,t4,-1
0x80100034:     ffd014e3        bne     zero,t4,0x8010001c
0x80100038:     00008067        ret
```

3. 执行程序：

```
>> G
addr: 0x80100000

elapsed time: 0.000s
```

4. 查看结果



3. 打印所有可打印字符

汇编代码：

```
    # 输出编码范围由 0x21 到 0x7E
    li a0, 0x21
    li t2, 0x7F
    # 接下来要调用 WRITE_SERIAL 函数
    # 调用函数的时候会将返回地址写入 ra，所以要预先保存 ra 中的地址到栈上
    addi sp, sp, -16
    sw ra, 12(sp)

print_loop:
    # 调用 WRITE_SERIAL 函数，打印 a0 对应的字符
    jal ra, WRITE_SERIAL
    # 将 a0 加 1，对应下一个字符
    addi a0, a0, 1
    # 如果 a0 不等于 t2，则继续打印
    bne a0, t2, print_loop

    # 完成打印后需要恢复栈状态和 ra
    lw ra, 12(sp)
    addi sp, sp, 16
    # 返回 kernel
    jr ra

# 打印函数
```

```
WRITE_SERIAL:
    # 串口地址是 0x10000000
    li t0, 0x10000000

    # 轮询串口状态（0x10000005）
.TESTW:
    lb t1, 5(t0)
    # 判断是否可写
    andi t1, t1, 0x20
    beq t1, zero, .TESTW

.WSERIAL:
    # 向终端（0x10000000）输出 a0 寄存器中的最低字节
    sb a0, 0(t0)
    # 跳转回 ra 中的地址
    jr ra
```

执行过程：

1. 从文件加载程序：

```
>> F
>>file name: /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/ascii.s
>>addr: 0x80100000
reading from file /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/ascii.s
[0x80100000] # 输出编码范围由 0x21 到 0x7E
[0x80100000] li a0, 0x21
[0x80100004] li t2, 0x7F
[0x80100008] # 接下来要调用 WRITE_SERIAL 函数
[0x80100008] # 调用函数的时候会将返回地址写入 ra，所以要预先保存 ra 中的地址到栈上
[0x80100008] addi sp, sp, -16
[0x8010000c] sw ra, 12(sp)
[0x80100010]
[0x80100010] print_loop:
[0x80100010] # 调用 WRITE_SERIAL 函数，打印 a0 对应的字符
[0x80100010] jal ra, WRITE_SERIAL
[0x80100014] # 将 a0 加 1，对应下一个字符
[0x80100014] addi a0, a0, 1
[0x80100018] # 如果 a0 小于 t2，则继续打印
[0x80100018] bne a0, t2, print_loop
[0x8010001c]
[0x8010001c] # 完成打印后需要恢复栈状态和 ra
[0x8010001c] lw ra, 12(sp)
[0x80100020] addi sp, sp, 16
[0x80100024] # 返回term
[0x80100024] jr ra
[0x80100028]
```

```
[0x80100028]
[0x80100028] # 打印函数
[0x80100028] WRITE_SERIAL:
[0x80100028] # 串口地址是 0x10000000
[0x80100028] li t0, 0x10000000
[0x8010002c]
[0x8010002c] # 轮询串口状态（0x10000005）
[0x8010002c] .TESTW:
[0x8010002c] lb t1, 5(t0)
[0x80100030] # 判断是否可写
[0x80100030] andi t1, t1, 0x20
[0x80100034] beq t1, zero, .TESTW
[0x80100038]
[0x80100038] .WSERIAL:
[0x80100038] # 向终端（0x10000000）输出 a0 寄存器中的最低字节
[0x80100038] sb a0, 0(t0)
[0x8010003c] # 跳转回 ra 中的地址
[0x8010003c] jr ra
```

2. 查看写入的程序：

```
>> U
addr: 0x80100000
num: 64
0x80100000:     02100513        li      a0,33
0x80100004:     07f00393        li      t2,127
0x80100008:     ff010113        addi    sp,sp,-16
0x8010000c:     00112623        sw      ra,12(sp)
0x80100010:     018000ef        jal     0x80100028
0x80100014:     00150513        addi    a0,a0,1
0x80100018:     fe751ce3        bne     a0,t2,0x80100010
0x8010001c:     00c12083        lw      ra,12(sp)
0x80100020:     01010113        addi    sp,sp,16
0x80100024:     00008067        ret
0x80100028:     100002b7        lui     t0,0x10000
0x8010002c:     00528303        lb      t1,5(t0)
0x80100030:     02037313        andi    t1,t1,32
0x80100034:     fe030ce3        beqz    t1,0x8010002c
0x80100038:     00a28023        sb      a0,0(t0)
0x8010003c:     00008067        ret
```

3. 执行程序：

```
>> G
addr: 0x80100000
!"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\]^_`abcdefghijklmnopqrstuvwxyz{|}~
elapsed time: 0.000s
```

4. 求fibonacci数列的第60项

汇编代码：

```
    # 执行 58 次循环
    li a0, 58
    # t0 保存旧值低位，t1 保存旧值高位
    li t0, 1
    li t1, 0
    # t2 保存新值低位，t3 保存新值高位
    li t2, 1
    li t3, 0
    # t4 保存刚刚算出来的值的低位，t5 保存高位
.L1:
    # 计算 fib(i)
    # 先进行低位相加
    add t4, t0, t2
    # 判断是否由进位，如果有进位，则t4 < t6，此时 t0 = 1
    sltu t6, t4, t0
    # 进行高位相加
    add t5, t1, t3
    # 再加上进位
    add t5, t5, t6
    # 保存结果
    addi t0, t2, 0
    addi t1, t3, 0
    addi t2, t4, 0
    addi t3, t5, 0
    # 剩余次数减一
    addi a0, a0, -1
    bne zero, a0, .L1
```

```
    # 循环结束，将结果保存到指定内存地址
    # 采用小端序
    li a1, 0x80400000
    sw t4, 0(a1)
    sw t5, 4(a1)

    jr ra


```

执行过程：

1. 加载程序：

```
>> F
>>file name: /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/fib60.s
>>addr: 0x80100000
reading from file /home/hhr/3_autumn/Computer_organization/Computer_organization_personal/lab1/fib60.s
[0x80100000] # 执行 58 次循环
[0x80100000] li a0, 58
[0x80100004] # t0 保存旧值低位，t1 保存旧值高位
[0x80100004] li t0, 1
[0x80100008] li t1, 0
[0x8010000c] # t2 保存新值低位，t3 保存新值高位
[0x8010000c] li t2, 1
[0x80100010] li t3, 0
[0x80100014] # t4 保存刚刚算出来的值的低位，t5 保存高位
[0x80100014] .L1:
[0x80100014] # 计算 fib(i)
[0x80100014] # 先进行低位相加
[0x80100014] add t4, t0, t2
[0x80100018] # 判断是否由进位，如果有进位，则t4 < t6，此时 t6 = 1
[0x80100018] sltu t6, t4, t0
[0x8010001c] # 进行高位相加
[0x8010001c] add t5, t1, t3
[0x80100020] # 再加上进位
[0x80100020] add t5, t5, t6
[0x80100024] # 保存结果
[0x80100024] addi t0, t2, 0
[0x80100028] addi t1, t3, 0
[0x8010002c] addi t2, t4, 0
[0x80100030] addi t3, t5, 0
[0x80100034] # 剩余次数减一
[0x80100034] addi a0, a0, -1
```

2. 运行程序：

```
>> G
addr: 0x80100000

elapsed time: 0.001s
```

3. 查看结果：

```
>> D
addr: 0x80400000
num: 8
0x80400000: 0x6c8312d0
0x80400004: 0x00000168
```

拼接起来就是`0x00000168_6c8312d0`

思考题

1. 比较 RISC-V 指令寻址方法与 x86 指令寻址方法的异同

相同点：

- 都支持立即数寻址，将操作数直接包含在指令中，如`addi a0, a0, 1`和`add eax, 1`
- 都支持寄存器寻址，将操作数存放在寄存器中，如`add a0, a0, a1`和`add eax, ebx`
- 都支持寄存器相对寻址，即将寄存器的值和偏移量相加作为地址进行访存，如：`lw a0, 12(a1)`和`mov eax, [ebx+12]`
- 都支持 **PC 相对的控制转移。**例如，RISC-V 的`beq`、`jal`和 x86 的相对分支、跳转指令，都可以根据指令位置与偏移量确定跳转目标。

不同点：

- RV32I 的访存地址采用“基址寄存器 + 12 位有符号偏移”。算术、逻辑运算在寄存器中完成，内存数据通过加载和存储指令访问。
- x86 的内存寻址更灵活，支持“基址 + 变址×比例 + 偏移”，例如`[ebx+ecx*4+8]`，而且部分算术指令可以直接使用内存操作数。

2. 阅读监控程序，列出监控程序的 19 条指令，请根据自己的理解对用到的指令进行分类，并说明分类原因。

可以按照指令的功能进行分类：

- 算术运算指令：`ADD`和`ADDI`，均用于完成加法计算。
- 逻辑运算指令：`AND`、`ANDI`、`OR`、`ORI`和`XOR`，用于完成按位与、或、异或等逻辑运算。
- 移位指令：`SLLI`和`SRLI`，分别用于逻辑左移和逻辑右移，可以调整数据的位位置。
- 访存指令：`LB`、`LW`、`SB`和`SW`，用于在寄存器与内存之间传输字节或字数据。
- 条件分支指令：`BEQ`和`BNE`，根据两个寄存器的值是否相等决定是否跳转，用于条件判断和循环控制。
- 无条件跳转与链接指令：`JAL`和`JALR`，用于跳转，并可保存返回地址，支持函数调用和返回。
- 常量与地址构造指令：`LUI`和`AUIPC`，前者用于构造高位立即数，后者用于构造相对于当前 PC 的地址。

3. 结合 term 源代码和 kernel 源代码说明 term 是如何实现用户程序计时的

主要的流程如下：

```
发送 G 和入口地址
        ↓
收到 Kernel 的开始标志 0x06
        ↓
记录开始时间，持续接收程序输出
        ↓
收到 Kernel 的结束标志 0x07
        ↓
计算时间差，显示用时
```

也就是 kernel 主动向 term 通知程序的开始和结束，term 收到开始标志之后开启计时，等待接收到结束标志后停止计时

计时逻辑主要位于 `term.py` 的 `run_G()`，Kernel 对应的代码位于 `shell.S` 的 `.OP_G` 和 `.USERRET2`。

1. 发送运行请求

```
outp.write(b'G')
outp.write(int_to_byte_string(addr))
```

kernel 收到之后进入 `.OP_G` 分支：

```
li a0, SIG_TIMERSET     # 0x06
jal WRITE_SERIAL
```

kernel在跳转到用户程序之前会向串口写入一个 `0x06` 作为开始标志

2. 接收到开始标志后，记录主机时间

```
ret = inp.read(1)

if ret != b'\x06':
    print("start mark should be 0x06")

time_start = timer()
```

term 一旦接收到开始标志，就会调用 `timer()` 获取当前时间

3. 运行期间正常接收并输出字符

```
while True:
    ret = inp.read(1)

    if ret == b'\x07':
        break

    output_binary(ret)
```

当读取到结束标志的时候才会结束，其余字符都是正常读取和输出

4. 收到结束标志后计算时间差

用户程序执行 `jr ra` 返回后，Kernel 的 `.USERRET2` 会：

```
保存用户寄存器
    ↓
恢复内核栈
    ↓
发送 0x07
    ↓
返回 SHELL
```

```
.USERRET2:
    // 定位用户空间寄存器备份地址
```

```
    la ra, uregs

    // 保存用户寄存器，此处省略具体指令
    // ……省略

    // 重新获得当前监控程序栈顶指针
    LOAD sp, TF_ksp(ra)
    mv a0, ra
    la ra, .USERRET2
    STORE ra, TF_ra(a0)

    // 发送停止计时信号
    li a0, SIG_TIMETOKEN
    // 告诉终端用户程序结束运行
    jal WRITE_SERIAL

    j .DONE
```

Term 收到 `0x07` 后，结束接收循环，并执行：

```
elapse = timer() - time_start
print('elapsed time: %.3fs' % (elapse))
```

得到程序运行用时

4. 说明 kernel 是如何使用串口的

Kernel 通过内存映射 I/O 的方式访问串口，即使用访存指令读写串口设备的寄存器。在本实验的配置中，串口数据寄存器的地址是 `0x10000000`，状态寄存器的地址是 `0x10000005`。这些地址对应硬件设备，读写它们会触发串口操作。

串口初始化由 `init.S` 完成，具体的收发功能在 `utils.S` 中实现，最基本的两个函数是 `WRITE_SERIAL` 和 `READ_SERIAL`。

`WRITE_SERIAL` 用于发送 `a0` 的低 8 位。它先读取串口状态寄存器，并通过与 `0x20` 进行按位与运算，检查发送保持寄存器是否为空。如果不可写，就不断读取状态，进行忙等待；如果可写，就使用 `sb` 将 `a0` 的低 8 位写入数据寄存器，由串口设备发送出去。

```
WRITE_SERIAL:                       // 写串口：将a0的低八位写入串口
    li t0, COM1
.TESTW:
    lb t1, %lo(COM_LSR_OFFSET)(t0)  // 查看串口状态
    andi t1, t1, COM_LSR_THRE       // 截取写状态位
    bne t1, zero, .WSERIAL          // 状态位非零可写进入写
    j .TESTW                        // 检测验证，忙等待
.WSERIAL:
    sb a0, %lo(COM_THR_OFFSET)(t0)  // 写入寄存器a0中的值
    jr ra
```

`READ_SERIAL` 用于接收一个字节。它读取状态寄存器，并通过与 `0x01` 进行按位与运算，检查是否有可读取的数据。如果没有，就继续忙等待；如果有，就使用 `lb` 读取数据寄存器，将收到的字节放入 `a0` 的低 8 位。

```
READ_SERIAL:                            // 读串口：将读到的数据写入a0低八位
    li t0, COM1
.TESTR:
    lb t1, %lo(COM_LSR_OFFSET)(t0)
    andi t1, t1, COM_LSR_DR             // 截取读状态位
    bne t1, zero, .RSERIAL              // 状态位非零可读进入读
    j .TESTR                            // 检测验证
.RSERIAL:
    lb a0, %lo(COM_RBR_OFFSET)(t0)
    jr ra
```

在这两个函数的基础上，Kernel 还实现了多字节数据和字符串的收发。`WRITE_SERIAL_WORD` 和 `READ_SERIAL_WORD` 用于收发 32 位数据，按低字节在前的顺序传输四个字节；`WRITE_SERIAL_XLEN` 和 `READ_SERIAL_XLEN` 用于收发一个机器字长的数据，在 RV32 下传输 4 字节，在 RV64 下传输 8 字节。`WRITE_SERIAL_STRING` 则逐字节发送字符串，直到遇到字符串结束符 `'\0'`。

因此，Kernel 与 Term 之间的命令、地址、内存数据以及运行状态标志，最终都通过这些串口读写函数进行传输。

5. 请问 term 如何检查 kernel 已经正确连入，并分别指出检查代码在 term 与 kernel 源码中的位置

Term 通过“发送探测命令并检查返回值”的方式，判断 Kernel 是否已经正确连入：Term 向 Kernel 发送 `W`，并等待其返回字长。如果收到预期的 `4` 或 `8`，说明 Kernel 能够接收命令并作出正确响应，Term 才会进入命令主循环。

Term 端的检查代码位于 term.py 的 Main() 函数：

```
# probe xlen
outp.write(b'W')
while True:
    xlen = ord(inp.read(1))
    if xlen == 4:
        print('running in 32bit, xlen = 4')
        arch = 'rv32'
        break
    elif xlen == 8:
        print('running in 64bit, xlen = 8')
        arch = 'rv64'
        break
    elif xlen < 20:
        print('Got unexpected XLEN: {}'.format(xlen))
        sys.exit(1)
MainLoop()
```

其中，`outp.write(b'W')` 发送探测命令，`inp.read(1)` 等待并读取一个响应字节。收到 `4` 或 `8` 时，Term 确定目标架构并退出检查循环，随后进入 `MainLoop()`，开始接受用户命令。如果收到其他小于 `20` 的值，则认为返回的字长不符合预期，打印错误信息并退出；对于大于等于 `20` 的字节，代码会继续读取。如果一直没有响应，程序就会等待在读取操作处，无法进入命令主循环。

Kernel 端对应的代码位于 shell.S 的 SHELL 命令分发部分。Kernel 首先通过 `READ_SERIAL` 接收命令，再判断命令类型。由于 `W` 不属于 `R`、`D`、`A`、`G`、`T`，因此会进入默认分支：

```
// 错误的操作符，输出 XLEN，用于区分 RV32 和 RV64
li a0, XLEN
// 把 XLEN 写给 term
jal WRITE_SERIAL
j .DONE
```

该分支将 `XLEN` 通过串口发送给 Term。在这份代码中，`XLEN` 表示字长的字节数，RV32 返回 `4`，RV64 返回 `8`，恰好对应 Term 检查的两个合法值。

因此，**Term 判断 Kernel 正确连入的关键，是收到对 `W` 命令的预期响应**